

Lab1：DataLab 实验报告

一、基本信息

项目	配置
实验环境	WSL2, Ubuntu 24.04, x86-64
编译器	GCC 13.3.0
编译参数	-O -Wall -fwrapv -m32
实现文件	bits.c，共 19 个函数

二、运行截图与测试结果

先清理旧程序，再按实验提供的 Makefile 重新编译：

```
make clean
make all
```

编译成功，生成 btest、fshow 和 ishow

1. 规则检查：check_ops.py

检查各函数的运算符使用和操作数限制：

```
./check_ops.py bits.c
```

19 个函数均通过，输出末行为：

```
All 19 functions passed operator checks.
```

```
will@Mention:~/ICS/Lab1-DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 37/48 operators
mul5Sat: 29/30 operators
classifyAdd3: 19/52 operators
floatScaleThreeHalves: 36/60 operators
floatRoundEven: 27/65 operators
float_i2f: 32/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
❖ will@Mention:~/ICS/Lab1-DataLab$
```

2. 正确性检查: btest

检查各函数的计算结果:

```
./btest
```

19 个函数的 Errors 均为 0, 输出末行为:

```
Total points: 110/110
```

```

● will@Mention:~/ICS/Lab1-DataLab$ ./btest
Score    Rating  Errors  Function
  1         1       0    signMask
  2         2       0    bitXor
  3         3       0   negativePart
  4         4       0   copyByteWithin
  4         4       0   logicalShift
  4         4       0   swapNibblePairs
  4         4       0 secondLowestZeroBit
  5         5       0    oddParity
  5         5       0 rotateRightBits
  5         5       0 roundEvenPow2
  5         5       0 midpointTowardFirst
  7         7       0 isBetweenEitherOrder
  7         7       0    mul5Sat
  7         7       0   classifyAdd3
  7         7       0 floatScaleThreeHalves
 10        10       0 floatRoundEven
 10        10       0 float_i2f
 10        10       0    bitCount
 10        10       0   bitReverse

Total points: 110/110
❖ will@Mention:~/ICS/Lab1-DataLab$

```

19 个函数均满足规则限制，并通过实验提供的正确性测试。

三、函数实现思路

P1. signMask

- **思路：**将 1 左移 31 位，得到仅最高位为 1 的掩码。
- **关键表达式：** $1 \ll 31$ ，对应位模式为 0x80000000。

P2. bitXor

- **思路：**对应位不同时结果为 1，也就是排除“同时为 1”和“同时为 0”两种情况。
- **关键表达式：** $\sim(x \& y) \& \sim(\sim x \& \sim y)$ ，只使用题目允许的 $\sim$ 和 $\&$ 。

P3. negativePart

- **思路：**用符号掩码控制是否保留取负结果。
- **符号处理：** $x \gg 31$ 在负数时为全 1，在非负数时为全 0；与 $\sim x + 1$ 相与，分别得到相反数或 0。
- **边界处理：** `INT_MIN` 取负后仍保留原位模式，符合实验的 32 位位向量规则。

P4. copyByteWithin

- **思路：**提取源字节，清空目标字节，再将源字节填入目标位置。
- **掩码与位移：** $src \ll 3$ 和 $dst \ll 3$ 将字节编号换算为位偏移；用 `0xFF` 提取源字节，用 $\sim(0xFF \ll dst8)$ 清空目标字节。
- **边界处理：**先保存源字节，因此源、目标编号相同时也能保留原值。

P5. logicalShift

- **思路：**先算术右移，再清除高位补入的符号位。
- **掩码构造：** $\sim(((1 \ll 31) \gg n) \ll 1)$ 保留低 $32-n$ 位，与 $x \gg n$ 相与即可。
- **边界处理：** $n=0$ 时掩码为全 1； $n=31$ 时仅保留最低位，均不会移位 32 位。

P6. swapNibblePairs

- **思路：**同时交换每个字节中的高、低四位。
- **掩码与位移：**构造 `0x0F0F0F0F`，提取低四位后左移 4 位；将原数右移 4 位，再用同一掩码提取原高四位，最后合并。

P7. secondLowestZeroBit

- **思路：**先取反，将寻找第二低的 0 转化为寻找第二低的 1。
- **位运算处理：** $zeros \& (zeros + \sim 0)$ 清除最低的 1；再用 $rest \& (\sim rest + 1)$ 提取剩余的最低 1。
- **边界处理：**原数不足两个 0 位时，`rest` 为 0，结果自然为 0。

P8. oddParity

- **思路：**通过逐层异或，将全部位的奇偶性汇总到最低位。
- **实现过程：**依次与右移 16、8、4、2、1 位的结果异或，最后返回 $!(x \& 1)$ 。1 的个数为偶数时返回 1，为奇数时返回 0。

P9. rotateRightBits

- **思路：**把右端移出的低位放到高位，与逻辑右移部分合并。
- **位数处理：**`shift = n & 31` 得到实际右移位数；`leftShift = (~shift + 1) & 31` 得到对应左移位数。
- **边界处理：**清除算术右移产生的符号扩展；`n` 为 0 或 32 的倍数时，两边移位量均为 0，返回原值。

P10. roundEvenPow2

- **思路：**先加入舍入偏置，再清除低 `n` 位，得到最接近的 2^n 的倍数。
- **偏置构造：**半程值为 2^{n-1} ，偏置取“半程值减一，再加商的最低位”，即 `half + ~0 + ((x >> n) & 1)`。
- **中点处理：**恰好位于半程时，偶数商不进位，奇数商进位，实现就近取偶；其余情况按距离舍入。

P11. midpointTowardFirst

- **思路：**先求向下取整的均值，再按第一个参数的方向修正半整数。
- **防止溢出：**用 `(x & y) + ((x ^ y) >> 1)` 求均值，避免直接计算 `x+y`。
- **取整处理：**两数奇偶性不同时，均值位于两个整数中间；若 `x > y`，将向下取整的结果加 1。比较时，异号看符号位，同号看差值符号。

P12. isBetweenEitherOrder

- **思路：**排除 `x` 同时小于两个端点、或同时大于两个端点的情况，剩余情况都在闭区间内。
- **比较处理：**分别比较 `x` 与 `a`、`b`，异号时根据符号位判断，同号时根据差值符号判断，避免溢出影响大小关系。
- **边界处理：**大于关系中排除相等情况，因此 `x` 等于任一端点时返回 1，无须预先排列端点。

P13. mul5Sat

- **思路：**先判断输入是否处于乘 5 的安全范围，再选择正常乘积或饱和值。
- **范围构造：**将 `0x99` 拼接为 `0x99999999`，与最高位掩码异或得到上界 `0x19999999`；下界为其相反数。
- **溢出处理：**结合输入符号判断是否越界。未越界时返回 `(x << 2) + x`；正溢出返回 `INT_MAX`，负溢出返回 `INT_MIN`。

P14. classifyAdd3

- **思路：**分两次相加，分别记录溢出方向，再合并判断结果。
- **溢出判定：**第一次计算 `first = x + y`，用 `(~(x ^ y) & (x ^ first)) >> 31` 判断“同号相加、结果异号”；第二次对 `first + z` 做同样处理。

- **方向合并：**正溢出记为 1，负溢出记为 -1，无溢出记为 0。相反方向的两次溢出会抵消，所以不能仅凭中间一次溢出判断最终结果。

P15. floatScaleThreeHalves

- **思路：**拆出符号、阶码和尾数，用整数运算完成乘 3、除 2 和就近取偶。
- **分类处理：**NaN 和无穷大直接返回；非规格化数将尾数乘 3 后右移 1 位；规格化数补上隐含最高位后乘 3，乘积小于 2^{25} 时右移 1 位，否则右移 2 位并将阶码加 1。
- **舍入与边界：**比较被丢弃部分与半程值，超过半程或恰好半程且保留部分为奇数时进位。非规格化结果达到规格化范围时自然形成相应编码；阶码溢出返回同号无穷大，零保留原符号。

P16. floatRoundEven

- **思路：**根据阶码定位小数部分，清除小数位后按就近取偶决定是否进位。
- **分类处理：**NaN、无穷大及阶码不小于 150 的数直接返回；绝对值不超过 0.5 时返回同号零，介于 0.5 和 1 之间时返回同号 1。
- **舍入处理：**其余数的小数位数为 $150 - \text{exp}$ 。超过半程时进位；恰好半程时，仅当整数部分为奇数才进位，负数同样保留符号。

P17. float_i2f

- **思路：**确定符号和最高有效位位置，再构造阶码与尾数。
- **有效位处理：**设最高有效位的位置为 `highest`，阶码为 `highest + 127`。有效位不超过 24 位时左移对齐；超过时右移保留 24 位，并根据丢弃部分就近取偶。
- **特殊情况：**`x=0` 直接返回 0；绝对值用无符号变量保存以容纳 `INT_MIN` 的大小。若舍入产生第 25 位，则再右移 1 位并增加阶码。

P18. bitCount

- **思路：**分组并行计数，再逐层合并。
- **掩码构造：**构造 `0x55555555`、`0x33333333` 和 `0x0F0F0F0F`，依次统计每 2、4、8 位中的 1，再通过右移相加合并到 16 位和 32 位。
- **结果提取：**总数在 0 到 32 之间，用 `x & 0x3F` 取出最低 6 位即可。

P19. bitReverse

- **思路：**先反转每个字节内部的位，再颠倒四个字节的顺序。
- **掩码构造：**由 `mask4 = 0x0F0F0F0F` 得到 `mask2 = mask4 ^ (mask4 << 2)`，再得到 `mask1 = mask2 ^ (mask2 << 1)`。
- **分组交换：**依次交换相邻的 1 位、2 位和 4 位分组，最后用移位与掩码重排字节。操作数为 32/34，满足题目上限。

四、参考资料与辅助工具

- [课程 Lab1 实验文档](#)：报告要求与测试流程。
- [实验仓库 README](#)及 `bits.c` 题目注释：函数语义、合法运算和操作数限制。
- chatgpt：用于思路启发、WSL2 环境配置与测试、报告整理。